

S-Cup: A Security Protocol for Self-reconfiguration by clustering on Programmable Matter based Modular Robots

*Note: Sub-titles are not captured in Xplore and should not be used

Youssou Faye
LI3/Department of Computer Sciences
University Assane Seck of Ziguinchor
Ziguinchor, Senegal
yfaye@univ-zig.sn

Abdallah Makhouf
Femto-st/ Department- DISC
University Marie et Louis Pasteur
Montbeliard, France
abdallah.makhouf@univ-fcomte.fr

Serigne Mbacke Diene
LI3/Department of Computer Science
University Assane Seck of Ziguinchor
Ziguinchor, Senegal
s.diene20150547@zig.univ.sn

Mohammed Ouzzif
C3S/ESTC
Hassane II University of Casablanca
Casablanca, Morocco
ouzzif@gmail.com

Cheikhou Oumar Sow
LI3/Department of Computer Science
University Assane Seck of Ziguinchor
Ziguinchor, Senegal
co.s1@zig.univ.sn

Abstract—Programmable matter in the context of microrobots is like mobile microcomputers with size of a millimeter, which can move around each other, communicate to form different shapes. Current challenges are as much theoretical as technological. The main research challenges in computer science focus on clustering, synchronization time, decision-making, analysis of detected data, security and the autoconfiguration process, which today remains the most fundamental. When the number of modules is very high, the self-reconfiguration challenge becomes more crucial for applications requiring rapid transition between two forms. Clustering can enable parallel transitions by allowing geographically close modules to make intra-cluster transitions, thus reducing reconfiguration complexity, the number of transitions and the time required. Clustering-based autoconfiguration solutions are available, but they do not include the security aspect. In this paper, we propose a security protocol for clustering-based self-reconfiguration solutions. This solution relies on resource-efficient cryptographic mechanisms to implement a robust authentication solution that underpins a flexible key management mechanism coupled with a confidential data exchange of the resulting structure

Keywords— *Modular Robots, Programmable Matter, self-reconfiguration based on clustering, Lightweight Cryptography*

I. INTRODUCTION

Modular robots are designed with modules that connect together to form autonomous systems with variable morphology that can perform different functions or tasks. One of the potential applications of self-reconfigurable modular robots is programmable matter [1]. Programmable matter based on modular robots envisages materials built as an assembly of thousands of micro-scale, millimeter-sized robotic modules that can move around each other, communicate to give different shapes [2]. Application areas are diverse and include space shuttles, medical devices, search and rescue, etc..... Whereas conventional robots can only be adapted to a single application or purpose, modular robots can fulfil a wide range of missions [3]. Current challenges are as

much theoretical as technological, and turn around modularity, mechanical design, electronic design, control algorithms, software and communications protocols. The main research challenges in computer science focus most on clustering, synchronization time, decision-making, analysis of detected data, security and, finally, the autoconfiguration process, which today remains the most fundamental. [3,4,5,6]. Self-reconfiguration in modular robotics has several drawbacks: i) the reconfiguration process is very time-consuming, especially when the number of modules is very high, which is unsuitable for many programmable matter applications, especially mission-critical applications requiring rapid transition between two forms; ii) most work is dedicated to one type of modular robot and cannot be generalized to others; iii) the computational complexity in most models is high and unsuitable for limited resources. Clustering can enable transition parallelism by allowing geographically close modules to make intra-cluster transitions, thus reducing reconfiguration complexity, the number of transitions and the time required. It was demonstrated in [7] that cluster-based autoconfiguration can improve the execution time and communication efficiency. Most of the clustering solutions proposed for modular robots have limitations for application in the self-configuration of programmable materials based on modular robots. Some of them consider the partitioning problem independently of the task at hand, which is self-configuration. [8, 9, 10], others focus on the configuration of small sets of modules [11, 12] or ignore the size constraint, which is crucial for transforming groups of the initial shape into specific parts of the target shape requiring a fixed number of modules [13, 14]. Recently, highly advanced clustering solutions for programmable materials based on modular robots have been proposed to overcome these shortcomings by adapting to the scaling, cluster size constraints and distributed environment [5,8,15]. However, they do not take security into account. To our knowledge, only one work exists, and it focuses more on the current hardware and software configuration of the amoebot model [16] to propose a multi-stage authentication service, without taking into

account possible evolutions relating to the material as such, which, through its physical configuration, underpins the exchange and autoconfiguration protocols [17].

In this paper, we propose S-Cup, a security protocol for clustering-based autoconfiguration solutions by using lightweight cryptographic to implement mutual authentication, flexible key management and confidential data exchange for the formed structure. The key management protocol adapts to the autoconfiguration process according to the pattern of structure skeleton formation and/or cluster formation, the degree of connectivity and the sharing of common keys in the same neighborhood.

The rest of the paper is organized as follows. We set the context in section 2 and the studied background in section 3. Section 4 sets out the assumptions and presents the operating architecture of the modular robot system. The safety challenges are presented in Section 5 and the proposed solution in Section 6, followed by an analysis in Section 7 and a conclusion and outlook in Section 8.

II. CONTEXT

The classification of modular robotic systems is quite a difficult task, due to the number of existing projects and the diversity of their task, form, size or control system. Some modules are controlled by external computers. Others are directed by a control module or are completely autonomous. Nevertheless, all modules are equipped in the same way, both physically and logically. A security solution is highly dependent on system architecture and configuration when deployed on the lower layers. A security solution for programmable material must be contextualized according to its framework and application domain, which is strongly correlated to its architecture. This application framework depends on the physical nature of the programmable material. We can distinguish between programmable matter based on modular robots such as ATRON and Sambot, and programmable matter based on tiny module such as 3D Catoms (millimetric scale). Each material is characterized by a number of properties.

- its mobility: is ensured by an actuating mechanism; BlinkyBlock is static, while 2D Catoms and 3D Catoms are mobile.
- its degree of freedom of movement: 2D Catoms move by rolling over their fixed neighbors, while 3D Catoms move around their neighbors.
- its number of neighbors: BlinkyBlock can have up to 4 neighbors, 2D Catoms up to 6, 3D Catoms up to 12.
- its field of application: the BlinkyBlock is used to create interactive works of art, education and research.

In the first case, where self-reconfiguring modular robots can modify their general form to adapt to the environment or to a task or scenario, depending on the data detected and analyzed from the real environment in which they evolve. In the second case, this set of tiny modules where each module has communication and computing capacity. This is known as modular robot-based programmable matter. In the following, we focus on programmable matter based on modular robots..

III. BACKGROUND

A. Programmable matter vs WSN and IoT

Modular robots have similarities with Wireless Sensor Networks (WSN) and the Internet of Things (IoT), but also differences that have a major impact on security. Lightweight security schemes have been proposed for IoT, WSN and RFID [18, 19, 20] are not adequate because modular robot systems present environments and structuring that make it difficult to execute certain algorithms. Furthermore, security algorithms proposed for WSN and IoT would not be suitable for sensors on modular robots [21, 22, 23, 24]. These differences with WSNs and the IoT include: i) a very large number of modules or particles or cells; ii) particles are anonymous and do not have a unique identifier like sensors; iii) a highly dynamic topology during autoconfiguration; iv) neighbor-to-neighbor communication (no routing); v) modules are not connected to the Internet. In addition to these specific features, there are other challenges linked to the type of communication (neighbor-to-neighbor by contact, wireless), the self-organization process (started by one module or parallel by several modules) and the system architecture (computer-controlled, controlled by a leader module, autonomous).

B. Security analysis in programmable matter

Security in modular robot-based programmable matter is a real challenge. Due to their characteristics, security protocols in WSN, IoT and RFID are inadequate. Security issues are not well addressed. To the best of our knowledge, only the Protocol of Lightweight Security Embedded in an Architecture of Nanobots (PROLISAN) has been proposed [7], and its scope of application is limited to a category of modules. PROLISAN uses lightweight cryptographic mechanisms based on hash functions and symmetrical block ciphers to reduce memory storage and computation time. It provides two security services: authentication of modules when they connect to the system, and confidentiality of communications between modules to prevent unauthorized third parties from learning about system behavior and decision-making. Four versions are available, with different levels of security and authentication.

The first version provides a weak authentication service based on the fingerprint from the hash of the source code. It does not guarantee membership in the structure, replay old messages is possible because the same fingerprint will be used by all modules. Only the source code is needed to create an authentication message.

The second version provides an authentication and confidentiality based on hashing the code into two fingerprints: one is used for authentication and the other as a data encryption key for confidentiality. Just like the first version, it does not guarantee membership in the structure, replay old messages is always possible because the same fingerprint will be used by all modules. In addition, it requires more resources with two fingerprints. Unlike the first, its security is designed based on the hash of the first n lines for authentication and the last m ($n \neq m$) lines for key. This does not solve the problem highlighted in the first version since the hash function used will always produce the same size of fingerprints and the same code will always produce the same fingerprint. If the channel is not physically secure, it is possible to capture a fingerprint to authenticate or decrypt the messages. So it can work with communication by contact, the signal must not be accessible to neighbors, and the code must not be known to the general public. The third version redefines

authentication. For both authentication and confidentiality, it uses the hash of a line instead of a part of the source code as in version 2. Its security depends on the line generation mechanism based on the authentication number and a chaotic function to determine the lines to be hashed. However, even if the character of determining the line to be hashed is more or less random, the hash of the line is sent for authentication, so replay attack is always possible. If the chaotic function is known, it is necessary to have the source code to create an authentication message. In version 2, the same key was used by neighbors, while here we have a key for each pair of modules. This version is based on the fact that malicious nodes do not know the protocol, which is a limitation for public use. Furthermore, the authentication mechanism stipulates that the moving module authenticates itself to the fixed module, there is a problem when two modules are moving, each module can remain waiting (infinite idle listening).

The fourth version, an improvement on the third, solves the problem of module mobility by using module identities distributed by a leader. Thus, modules authenticate themselves through their ID, their Clock, which determines the part to be hashed. Module A authenticates B through ID_B, the hash of part of B's code H(Code_B), and the Clock_B that B sent to it. And through B's clock, A identifies the corresponding part of its code, which it hashes and verifies with the code received from B. This version uses the same encryption method as version 3. Although it regulates module movement, this version runs in a more restrictive environment since a leader and IDs are required, distributed a priori. This raises the question of contact-based communication for moving modules. If the clocks were synchronized, there would be no need to send them; sending a clock simply reveals the part of the code to be used for authentication.

IV. ASSUMPTIONS AND SYSTEM ARCHITECTURE

A. System assumptions

Here, we plan to study the security of clustering-based autoconfiguration of a programmable matter system based on modular robots that is very similar to 3D Catoms in that it will have the following properties:

- The system is distributed, and controlled by a computer or a leader module that can just play the role of giving the signal and shape to start the autoconfiguration process.
- In the initial form, the modules are divided into groups that will later form the goal form clusters.
- During the formation of goal-form, we distinguish three types of communication:
 - *Phase 1:* communication between CHs or inter-Clusters to form the skeleton of the structure
 - *Phase 2:* Communication between members of the same cluster or intra-cluster communication to shape the cluster which is part of the goal form.
 - *Phase 3:* Communication between any two neighboring modules in the structure to connect two clusters if necessary, or during structure operation.
- When the structure is formed, a module only communicates with its neighbors via one of its connectors or a wireless signal.
- A module can have as many neighbors as required.

- All modules are identical and execute the same source code.
- Once clusters have been formed, a module can communicate with a neighbor in a different cluster.
- Clocks are reset at the same time and each time modules receive the signal to start the autoconfiguration process.
- Modules have no identifier
- The structure will be formed from the cluster leaders, initially giving the skeleton.
- No secrets are shared a priori; everything will be built from 0.

B. System architecture

Modular robot-based systems are limited in terms of basic architecture, motion and communication. Current hardware and software technologies work on the basis of chain, lattice or hybrid architectures, sometimes supported by a sandbox with modules that rotate around their pivot neighbors or slide on the lattice, and that can communicate by contact. Here, we study the autoconfiguration process on more advanced architectures with greater freedom of movement and communication between modules. For a scalable safety protocol that will take into account future technological and software mutations in modular robot systems, we put ourselves in a context that takes into account the following assumptions:

- Modules that can communicate by contact and wirelessly
- Modules that can move on a trellis or on their neighbors, on a support on which they are placed or in the environment in which they evolve.
- Modules that can move in any direction with a total degree of freedom.
- Modules communicate only with their direct (one-hop) neighbors, which are limited in number.

Based on these assumptions, we consider an architecture where modules operate as a distributed system during self-reconfiguration. The system is formed in clusters, with a cluster leader (CH) in each cluster. Modules can communicate by contact or wirelessly. Clusters are made up of geographically close modules the figure "Fig.1". shows an architecture with wireless communication between CHs and contact communication between CHs and MCs (cluster members). We won't go into the process of electing cluster leaders, which can be based on criteria such as residual energy, geographical position, etc....

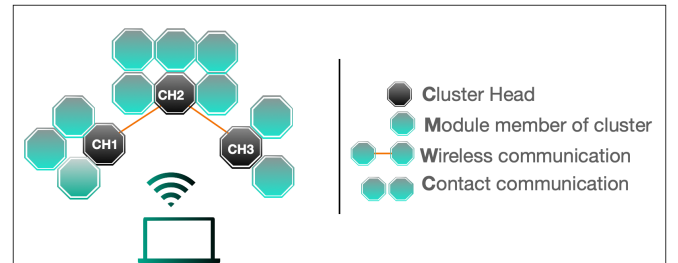


Fig. 1. Clustering System Architecture

The modules of the initial structure are divided into groups based on the goal form from. The security process begins really when the goal form clusters start. From the architecture shown in Fig.2, we move from an initial set of modules

arranged in grid form to a set of modules divided into groups of modules to form the goal form clusters. Each color represents a cluster, the three clusters each have three modules and the fourth has 6 modules. The diagram in Fig.3. shows us the transition from this set of modules divided into groups on a grid (the initial form) to the final or goal form (Cup) according to the following steps:

1) *Skeleton-forming phase or inter-cluster communication phase*: CHs position themselves in strategic locations that generate less movement to form the skeleton of the structure as shown in Fig.3.-(a), only CHs communicate with each other by contact or wirelessly. In our case, cluster 4 is automatically formed.

2) *Cluster formation phase or intra-cluster communication phase*: each cluster is formed by the transitions of CMs to give part of the goal shape as shown in Fig.3.-(b) and Fig.3.-(c).

3) *The operation phase*: once the structure has been formed, it becomes operational and modules need to communicate with their neighbors, whether or not they belong to the same cluster (Fig.3.-(c)).

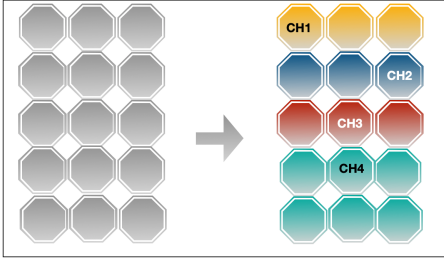


Fig. 2. Preliminary groups segmentation

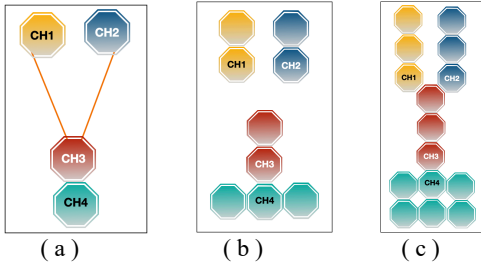


Fig. 3. Progressive self-reconfiguration

V. SECURITY CHALLENGES

The environment of programmable matter based on modular robots is very restrictive in terms of security.

- The main challenge lies in the limited resources of modules that are unsuitable for resource-intensive security mechanisms.
- The lack of identifiers, which complicates identification and authentication mechanisms.
- For public use, security solutions need a priori information sharing for a security pre-configuration phase, as in the case of access points, or some IoT networks, etc. For private use, this a priori security configuration can meet the user's needs. However, for public use of modular robots, it is virtually impossible to configure hundreds of thousands of modules.
- The use of contact communication would be more secure in the face of some attacks on the channel, but

we advocate more scalable modules that are sufficiently autonomous to be able to communicate by contact and via a wireless channel.

- Authentication is the major challenge as there is no central entity in the strict sense of the term to manage authentication. Modules have no identifiers. Hence, it is difficult to produce authentication through a peer-to-peer relationship.
- Confidentiality is ensured by encryption, which requires key management. The absence of a trusted third party makes it difficult to use asymmetric cryptography. And in the absence of a module identifier, sharing symmetrical keys without a third party poses more challenges in a fully distributed environment.
- Key management is a real challenge, in that it must be economical in terms of memory, computation and energy, to meet the very limited resources of the modules. As far as possible, key storage must be avoided, and communication to establish keys minimized. In addition, it must take place in a distributed environment, with no a priori secret sharing and no central entity.

VI. SECURE-CUP

We propose the Secure-Cup (S-Cup) a Security Protocol for Self-reconfiguration by clustering on Programmable Matter based Modular Robots to secure the clustering-based autoconfiguration process. S-Cup provides authentication, key management and confidentiality services during skeleton-formation phase, clusters formation phase and the operational phase. It is based on an authentication mechanism at the end of which, a peer key is established between two CHs during the first phase, a group key for each cluster during the second phase, and a global key for the whole structure during the operational phase. To reduce autoconfiguration time, we consider that the structure will be formed in parallel from the CHs, which will first perform a pairwise mutual authentication to build a skeleton of the connected structure.

Authentication involves a CH proving that it belongs to the structure by proving that it executes the same code as those in the structure. Since all modules execute the same code, and there is no trusted third party at the outset, it is difficult for two modules to authenticate in a peer-to-peer relationship. We distinguish between three types of authentications:

- Authentication between CHs, which is mutual between CHs to be connected during skeleton formation (Fig. 3-(a)).
- Mutual authentication between CHs and CMs in the same cluster during cluster formation (Fig. 3-(b)).
- Authentication between CH and CM, or CM and CM from different clusters, for inter-cluster connection. The notations in TABLE I will be used in the following.

A. Security of skeleton formation

During this phase, the CHs must mutually authenticate and establish either a pairwise key between neighboring CHs, or a single key shared by all CHs, depending on the skeleton

TABLE I. NOTATION

Symbols	Description
CH	One Cluster Head of the structure
iCH	Initial Cluster Head
CH _n	The Cluster Head n, n is an integer
n _i	Number once, i is an integer
T _s	Current sending time
T _r	Current reception time
L _i	Line i in the code
N _b	Number of lines code
H()	One-way hash function
⊕	Exclusive-or (XOR) operation
	Concatenation
→	Transmission
Δt	Transmission delay
≈	Approximate value to nearest integer

formation model. We distinguish between the serial skeleton formation model (i.e., starting from a single CH) and the parallel formation model (i.e., starting from multiple CHs simultaneously). The serial model allows for the generation of a single key for the entire skeleton, or even the whole structure, but with a longer transition time. In contrast, the parallel model results in a shorter formation time, with one key per pair of CHs

1) Serial skeleton formation model

The skeleton begins to form from an initial CH or iCH, which can be designated by election, by a computer or by a leader module according to criteria such as geographical position, for example. The skeleton starts with the iCH module, which will connect the first module by mutually authenticating. All clocks are synchronized at the start of the experiment, so that the same timer T is used.

A CH that initiates a connection specifies in the request the period T_s for sending the request, a nonce n_i that both identifies the request and is used to calculate the line to be hashed. The period T_s is the nearest integer value of the sending time noted t_s at clock level, divided by ten: T_s ≈ (t_s/10). This makes it possible to correct small clock desynchronization offsets.

When the experiment is started and the initial CH iCH in place. The iCH is the CH3 for our skeleton in Fig.3.-(a) due to its centric position.

- A CH_n (CH4 for example), sends an authentication request to the iCH to prove that it belongs to the skeleton. It generates a nonce n₀. An authentication request is identified by its composition; no need to tell the iCH that it's a first request for skeleton formation, as it knows it's the initial iCH and is not linked to any module. CH_n calculates: n₁ = H(n₀); the authentication fingerprint K₀ by applying the hash function H() on the line L(n₀ modulo N_b) of the source code, i.e. K₀=H(L(n₀ modulo N_b)); x=T_s⊕n₀ to protect nonce n₀ then sends n₁ and (x||K₀) to the iCH. The fingerprint n₁ in the request enables the iCH to check that the clocks are synchronized.
- After receiving the message, iCH examines the message reception time T_s at clock level, calculates T_s ≈ ((T_r-Δt)/10), then calculates n₀'=x ⊕T_s, and checks whether n₁=H(n₀'). If so, synchronization is good. iCH must now authenticate CH_n by checking

whether K₀=H(L(n₀' modulo N_b)). If so, iCH launches the key generation challenge by generating a nonce n₂, calculates x₁=n₂⊕n₀, generates the key K₁=H(L(n₂ modulo N_b)) and sends x₁ to CH_n.

- CH_n calculates n₂=x₁⊕n₀, and in turn generates the key K₁=H(L(n₂ modulo N_b)). So iCH and CH_n have authenticated each other and share the key K₁, as shown in Algorithm 1. In this case may be CH4 first, as it is closer to CH3.
- In the same way, the protocol takes place between CH1 and iCH, between CH2 and iCH except that at phase SF-1, the nonces n₀ and n₁ chosen by each CH_n will be different. However, CH_i always keeps its nonce n₂ at phase SF-4, so that it can generate the same key K₁ for everyone. This results in the key scheme shown in Fig.4-(a).

Algorithm 1: Authentication for skeleton formation

SF-1 : CH _n	Chooses n ₀ ,
: CH _n	Computes n ₁ = H(n ₀)
: CH _n	Computes K ₀ =H(L(n ₀ modulo N _b))
: CH _n	Computes x=T _s ⊕n ₀
SF-2 : CH _n → iCH	n ₁ , (x K ₀)
SF-3 : iCH	Computes T _s ≈ ((T _r -Δt)/10)
: iCH	Drift n ₀ '= x ⊕T _s
: iCH	Verify if n ₁ = H(n ₀ ')
: iCH	Verify if K ₀ =H(L(n ₀ modulo N _b))
SF-4 : iCH	Chooses n ₂
: iCH	Computes x ₁ =n ₂ ⊕n ₀
: iCH	Computes K ₁ = H(L(n ₂ modulo N _b))
: iCH → CH4	x ₁
SF-5 : CH _n	Computes n ₂ =x ₁ ⊕n ₀
: CH _n	Computes K ₁ =H(L(n ₂ modulo N _b))

For one unique key in the skeleton, the CH_n already linked CH1, CH2 and CH4 will play the same role as the initial CH (CH3) with the other CHs not yet linked, as would be the case with a possible CH5 belonging to the skeleton. For a link between CH1 and CH2, the first to receive the request will play the role of iCH, which will always lead to a one-key scheme, as shown in figure 4.-(b).

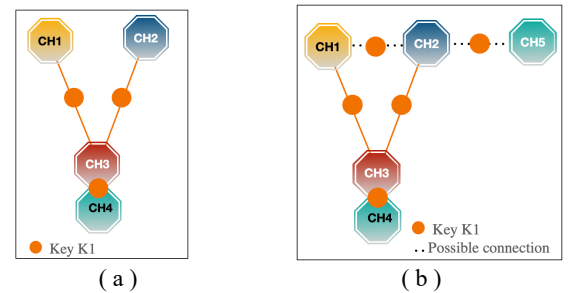


Fig. 4. Single Key model

2) Parallel Skeleton Formation

Parallel training has the advantage of reducing skeleton formation time, especially when the number of CHs is high. From Algorithm 1, we can move towards a scheme where each pair of CHs shares a common key. Thus, CHs establish keys in pairs using Algorithm 1, the connection initiator first authenticates as CH_n in phase SF-2, and the connection recipient authenticates by issuing a challenge, as iCH does in phase SF-4.. In our example, Algorithm 1 can be run in

parallel between CH3 and CH4 at the same time as between CH1 and CH2 to create keys K1 and K2 respectively, then between CH3 and CH1 and then between CH3 and CH4 to create keys K3 and K4 as it shown in Fig. 5. Key diversity comes from the fact that a CH chooses new nonces each time Algorithm 1 is run.

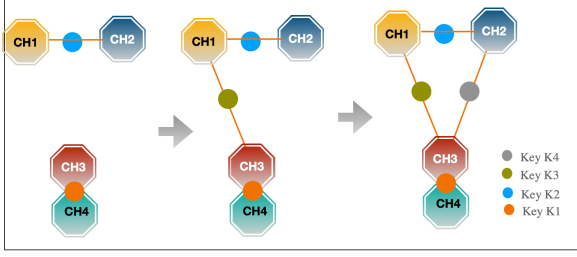


Fig. 5. Parallel pairwise key establishment within the skeleton

3) Clustering Phase

After the skeleton formation phase, comes the cluster formation phase, which proceeds identically regardless of whether the skeleton was formed serially or in parallel. Each CH will create a unique key for its entire cluster, still based on Algorithm 1, where it plays the same role as the iCH for the MCs directly connected to it.

Depending on the nonce that the CH either reuses or generates during phase SF-4, we identify three possible scenarios:

- *Case 1:* The CH reuses, in phase SF-4, the same nonce n_2 that was used during serial skeleton formation.
- *Case 2:* The CH uses a different nonce n_2' in phase SF-4, distinct from the n_2 used during serial skeleton formation.
- *Case 3:* The CH reuses, in phase SF-4, one of the nonces used with its neighboring CHs or generates a new nonce instead of n_2 during parallel skeleton formation.

The MCs directly connected to the CH will then, in turn, act as iCHs in Algorithm 1 toward the MCs that are not directly connected to the CH, which they must connect using the same nonce n_2 received from the CH during phase SF-4. Figure 6 illustrates the security protocol between a CH directly connecting to MC1, which in turn directly connects to MC2, corresponding to Case 3.

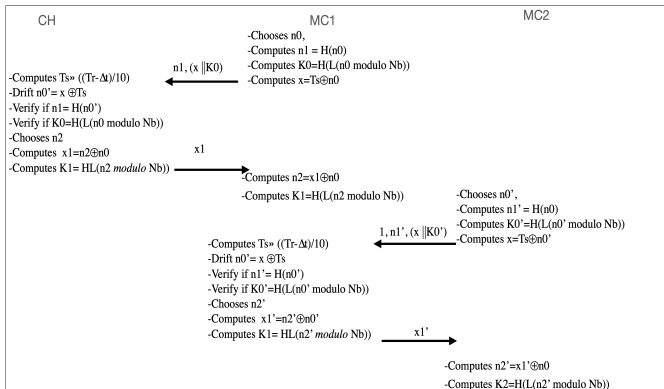


Fig. 6. Security Protocol between CH-MC1-MC2

VII. SECURITY ANALYSIS

A. Security

In this section, we provide an analysis of the security services provided and a theoretical evaluation of performance and robustness against attacks. The most difficult thing in this environment, with no security channel or trusted third party, is to share a key without any a priori shared information. Here, we first use clock synchronization at the start of each phase. We use an approximation to eliminate any offset with a certain degree of uncertainty. Calculating the transmission clock divided by 10 and rounded off to the nearest integer allows us to correct for uncertainties of -5 to +5 of the normal value. For example, for an offset of 8 units between the receiver and transmitter clocks (i.e. 96 for the receiver and 104 for the transmitter), the receiver finds $T_s = 96/10 = 9.6$ or 10, which is also equivalent to the approximate value of the transmitter clock, $T_s = \frac{104}{10} = 10.4$ or 10.

In Algorithm 1, the nonce n_0 used is never disclosed and enables a new module to authenticate itself and generate the key that will be shared between iCH and CH_n , while the fingerprint n_1 derived from n_0 will be used to verify synchronization and identify the request. If phase SF3 of Algorithm 1 is successfully completed, iCH will have authenticated CH_n by proving that it has the same code. In phase SF-5, iCH challenges CH_n through a request during which CH_n is asked to prove by another line that it has the same code as iCH, otherwise it will not be able to share a common key with iCH. If CH_n succeeds in generating the expected key, it will also have mutually authenticated iCH.

The replay attack, which is the most critical, is impossible for several reasons. A repeated message will never be synchronized, as it will be delayed and the T_s calculation will fail. Without n_0 , the message sent in phase SF-2 of Algorithm 1 cannot be generated.

Furthermore, the protocol offers a wide range of key generation methods, from one key shared by the entire network to one key per module pair. Noted that, for a unique key, the number of hash operations required to generate the key will be greater, and the skeleton formation phase will take longer.

However, the network shares a global key, which means that all the structure's communications can be protected without any additional mechanism during the operational phase. On the other hand, for a key per pair of CHs, the number of hash operations required to generate the keys is reduced, and the skeleton formation time is also reduced, especially when the number of CHs is high. However, the probability of two modules close to the structure sharing a key becomes low, and for a connection between two clusters after their formation, the modules involved still have to authenticate each other, which requires an additional mechanism. After the authentication and key generation phase, two neighbors share at least one key and will be able to encrypt data in the operational phase. It is important to note that keys do not have to be stored, but rather generated. A module just needs to store the nonce n_0 , from which it can generate keys and possibly other nonces.

B. Overhead Cost Comparisons

We analyze and compare our proposed solution to show its complexity (computation, time, communication, storage) and its resistance to forgery and repetition attacks. Complexity is assessed for the formation of a secure link between two modules. Table II shows that S-cup offers better

safety than all 4 PROLISEAN versions, but consumes slightly more energy than versions 1 and 2. If we quantify the time taken for each hash, encryption or transmission operation, we'll be able to determine how long the autoconfiguration process will take. This will depend on the total number of links in the structure, as Algorithm 1 is executed on each link. We can model this system with graphs, where the nodes are the modules, the degree the neighbors. We can then calculate the number of links to know how many times Algorithm 1 will be executed. If v is the number of neighbors for each module and m the total number of modules, then the number of links is: $w = \frac{v.m}{2}$. The cost c is given by (1).

$$c = \frac{v.m}{2} (6TH + 2Tx + 4XOR) \quad (1)$$

So, for a structure to be formed, the overall time complexity will be linear with the number of links and will be $O(w)$.

TABLE II. OVERHEAD COMPARISONS

Solutions » Parameters	S-Cup	PROLISEAN			
		V1	V2	V3	V4
Space Complexity	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Time Complexity	$O(1)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$
Storage Complexity	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Transmission Complexity	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Number of keys	Pairwise	No	1	Pairwise	Pairwise
Authentication	Hight	Medium	Medium	Medium	Hight
Confidentiality	Hight	No	Low	Low	Hight
Replay again replay Attacks	Hight	No	No	Low	Medium
Protection again Forgery Attacks	Hight	No	No	Low	Medium
Overhead Cost	$6H+2x+4XOR$	$2H+2Tx$	$4H+2Tx$	$4H+4Tx$	$4H+4Tx$

TH : the time for performing a one-way hash function $h()$.

$TXOR$: the time for performing an XOR operation.

Tx : the delay time for the communication taken place in one-hops.

VIII. CONCLUSION

In this article, we proposed S-Cup, a Secure-Cup, a new protocol to secure the self-configuration process based on clustering, and at the same time the structure thus formed during its operational phase. The proposed protocol implements a robust mutual authentication mechanism, after which a key is generated to ensure the confidentiality of the messages exchanged. The protocol uses lightweight cryptographic mechanisms to implement adaptive key generation models in a highly constrained modular robot system environment. Security analysis shows that the protocol handles many types of attack, notably replay and fabrication attacks, which are the most frequent and difficult to counter in these environments. As a follow-up to this work, we will implement this protocol with different key usage models in VisibleSim to measure their efficiency in terms of energy consumption, memory and computation of cryptographic operations. We will also implement the Avispa simulator to confirm the verification of security properties.

REFERENCES

- [1] J. Bourgeois, et al.: Programmable matter as a cyber-physical conjugation. In: 2016 IEEE International Conference on Systems, Man, and Cybernetics (SMC), pp. 002942–002947. IEEE (2016)
- [2] P., Thalamy, B. Piranda, & J. Bourgeois, (2019). A survey of autonomous self-reconfiguration methods for robot-based

- programmable matter. Robotics and Autonomous Systems, 2019, vol. 120, p. 103242.
- [3] Ahmadzadeh and E. Masehian, “Modular robotic systems: Methods and algorithms for abstraction, planning, control, and synchronization,” Artificial Intelligence, vol. 223, pp. 27–64, 2015.
- [4] L. Mihaylova, T. Lefebvre, H. Bruyninckx, K. Gadeyne, and J.D. Schutter, “A comparison of decision making criteria and optimization methods for active robotic sensing,” in International Conference on Numerical Methods and Applications. Springer, 2002, pp. 316–324
- [5] J. Bassil, A. Makhoul, B. Piranda, J. Bourgeois: Distributed size-constrained clustering algorithm for modular robot-based programmable matter. ACM Trans. Auton. Adapt. Syst. 18(1), 1:1–1:21 (2023)
- [6] J. Bassil, B. Piranda, A. Makhoul, & J. Bourgeois, Distributed Configuration Recognition for 2D Lattice-Based Modular Robots. In :International Conference on Advanced Information Networking and Applications: Springer Nature Switzerland, 2025, April, p. 140-151..
- [7] Moussa, M., Piranda, B., Makhoul, A., & Bourgeois, J. (2021, April). Cluster-based distributed self-reconfiguration algorithm for modular robots. In International Conference on Advanced Information Networking and Applications (pp. 332-344). Cham: Springer International Publishing.
- [8] Mazdin, P., & Rinner, B. (2021). Distributed and communication-aware coalition formation and task assignment in multi-robot systems. IEEE Access, 2021, vol. 9, p. 35088-35100.
- [9] Dutta, A., Ufimtsev, V., Asaithambi, A., & Czarnecki, E. (2019). Coalition formation for multi-robot task allocation via correlation clustering. Cybernetics and Systems, 2019, vol. 50, no 8, p. 711-728.
- [10] Dutta, A., Dasgupta, R., Baca, J., & Nelson, C. A. (2015, January). Spanning Tree Partitioning Approach for Configuration Generation in Modular Robots. In FLAIRS (pp. 360-365).
- [11] Dutta, A., Ufimtsev, V., Asaithambi, A., & Czarnecki, E. (2019). Coalition formation for multi-robot task allocation via correlation clustering. Cybernetics and Systems, 2019, vol. 50, no 8, p. 711-728.
- [12] Cruz, N. B., Nedjah, N., & de Macedo Mourelle, L. (2017). Robust distributed spatial clustering for swarm robotic based systems. Applied Soft Computing, 2017, vol. 57, p. 727-737.
- [13] Bassil, J., Moussa, M., Makhoul, A., Piranda, B., & Bourgeois, J. (2020, October). Linear distributed clustering algorithm for modular robots based programmable matter. In 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (pp. 3320-3325).
- [14] Shao, J., Yang, Q., Zhang, Z., Liu, J., & Kramer, S. (2018). Graph clustering with local density-cut. In Database Systems for Advanced Applications: 23rd International Conference, DASFAA 2018, Gold Coast, QLD, Australia, May 21-24, 2018, Proceedings, Part 1 23 (pp. 187-202). Springer International Publishing.
- [15] A. Majed, H. Harb, A. Nasser, et B. Clement, « RUN: a robust cluster-based planning for fast self-reconfigurable modular robotic systems », Intell. Serv. Robot., vol. 16, no 1, p. 75-85, mars 2023, doi: 10.1007/s11370-023-00454-w.
- [16] Z. Derakhshandeh. 2017. Algorithmic Foundations of Self-Organizing Programmable Matter. Ph.D. Dissertation. Arizona State University
- [17] Z. Derakhshandeh. 2017. Algorithmic Foundations of Self-Organizing Programmable Matter. Ph.D. Dissertation. Arizona State University
- [18] [17] Hourany, E., Habib, B., Fountaine, C., Makhoul, A., Piranda, B., & Bourgeois, J. (2021). Prolisean: A new security protocol for programmable matter. ACM Transactions on Internet Technology (TOIT), 2021, vol. 21, no 1, p. 1-29.
- [19] [18] Victor Kathan Sarker, Tuan Nguyen Gia, Hannu Tenhunen, and Tomi Westerlund. 2020. Lightweight Security Algorithms for Resource-constrained IoT-based Sensor Nodes. In ICC 2020-2020 IEEE International Conference on Communications (ICC). IEEE, 1–7.
- [20] [19] Mohammad Wazid, Ashok Kumar Das, Vivekananda Bhat, and Athanasios V Vasilakos. 2020. LAM-CIoT: Lightweight authentication mechanism in cloud-based IoT environment. Journal of Network and Computer Applications 150 (2020), 102496.
- [21] [20] Zhang, Y., Parker, L. E., & Kambhampati, S. (2014, May). Coalition coordination for tightly coupled multirobot tasks with sensor constraints. In 2014 IEEE international conference on robotics and automation (ICRA) (pp. 1090-1097). IEEE.
- [22] Faye, Y., Guyennet, H., Niang, I., & Shou, Y. . Fast scalar multiplication on elliptic curve cryptography in selected intervals

suitable for wireless sensor networks. In : Cyberspace Safety and Security: 5th International Symposium, CSS 2013, Zhangjiajie, China, November 13-15, 2013, Proceedings 5. Springer International Publishing, 2013. p. 171-182.

[23] [22] Faye, Y., Niang, I., & Guyennet, H. (2019, March). New Node Distributed Access Control Scheme for Wireless Sensor Network. In : Proceedings of the 9th International Conference on Information Systems and Technologies. 2019. p. 1-4.

[24] [23] Soni, S., & Hussain, M. (2025). Light Weight Authentication of Devices in Wireless Sensor Network. Procedia Computer Science, 2025, vol. 252, p. 1005-1013.

[25] Saha, D. (2024). Improving IoT Security in Wireless Sensor Networks. Smart Internet of Things, Smart Internet of Things, 2024, vol. 1, no 4, p. 289-297. J. Clerk Maxwell, A Treatise on Electricity and Magnetism, 3rd ed., vol. 2. Oxford: Clarendon, 1892, pp.68-73.